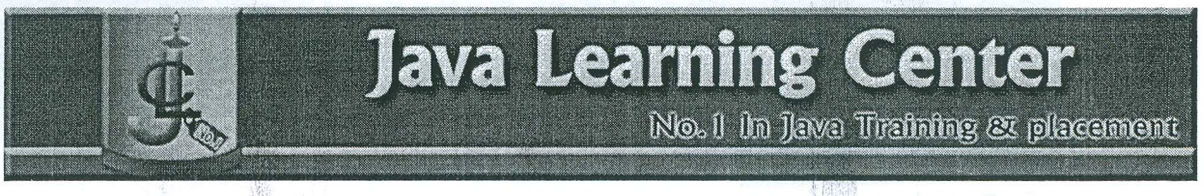
Java - 23



Java Learning Center

enum

Put into Knowledge

Doc Version - 1.0

Author

Srinivas Dande

Kamlesh Kumar Singh

Introduction

Lab1356.java

```
public class Lab1356{
public static void main(String[] args) {
Course c1=Course.JAVA;
System.out.println(c1);
System.out.println(Course.JDBC);
System.out.println(Course.SERVLETS);
System.out.println(Course.SPRING);
Course c2=Course.SPRING;
c2.setId(4);
c2.setName("SPRING");
c2.setFacultyName("Dande");
c2.setDuration(3);
c2.setFee(5000);
System.out.println(c2);
}
}

class Course {
public static Course JAVA=new
Course(1,"Java","DK",2,3000.0);
public static Course JDBC=new
Course(2,"JDBC","Sri",1,4000.0);
public static Course SERVLETS=new
Course(3,"SERVLETS","Nivas",1,4000);
public static Course SPRING=new Course();
private int id;
private String name;
private String facultyName;
private int duration;
private double fee;
Course(){}
public Course(int id, String name, String facultyName, int
duration, double fee) {
        this.id = id;
        this.name = name;
        this.facultyName = facultyName;
        this.duration = duration;
        this.fee = fee;
}
public void setId(int id) {
        this.id = id;
}
public void setName(String name) {
        this.name = name;
}
public void setFacultyName(String facultyName) {
        this.facultyName = facultyName;
}
public void setDuration(int duration) {
        this.duration = duration;
}
public void setFee(double fee) {
        this.fee = fee;
}
public String toString() {
        return
id+"\t"+name+"\t"+facultyName+"\t"+duration+"\t"+fee;
}
}
```

Lab1357.java

```
public class Lab1357 {
public static void main(String[] args){
Direction d1=null;
//d1=new Direction("EAST",0);
d1=Direction.EAST;
System.out.println(d1);
System.out.println(Direction.NORTH);
System.out.println(Direction.WEST);
System.out.println(Direction.SOUTH);
}}
class Direction{
public static Direction EAST = new
Direction("EAST",0);
public static Direction NORTH = new
Direction("NORTH",90);
public static Direction WEST= new
Direction("WEST",180);
public static Direction SOUTH = new
Direction("SOUTH",270);
static{
System.out.println("** Static Block in Direction class\n");
}
private String name;
private int angle;
private Direction(String name,int angle){
 this.name = name;
 this.angle = angle;
 System.out.println(" Direction(String,int) Cons");
}
public int getAngle(){
 return this.angle;
}
public String toString() {
 return name+"\t\t"+angle;
}
}
```

four constant

- Enum is keyword added in Java 5.
- It is used to define user define data type like class.
- Enum is a user defined data type whose fields consist of a fixed set of constants of that enum type.
- Enum is special type of class that extends java.lang.Enum.

```
public enum Direction {
        EAST,
        WEST,
        NORTH,
        SOUTH     //optionally can end with ";"
}
```

static final constant

- Here EAST, WEST, NORTH and SOUTH are final static constants of type Direction.
- Default constructor will be used by default to initialize member of enum.
- You can use argument constructor to initialize the state of enum types.
- You can use only private modifier with enum constructor.
- enum Direction {

```
        EAST(0), WEST(180), NORTH(90), SOUTH(270);
        private Direction(final int angle) {
                this.angle = angle;
        }
        private int angle;
        public int getAngle() {
                return angle;
        }
}
```

other modifier are not allowed

Enum Direction {
EAST (EAST, 0),
WEST (WEST, 90),
};

no need to write public static Direction

enum → by default extends java.lang.Enum

- You can override only toString() method from Object class in enum.
- Other methods of Object class are defined as final in java.lang.Enum, so can't be overridden.
- Enum Constant can be accessed using the following method:
  - public final String name()
- By default name will be initialized with enum constant only.

Lab1358.java

```
public class Lab1358 {
public static void main(String[] args) {
Course c1=Course.JAVA;
System.out.println(c1);
System.out.println(Course.JDBC);
System.out.println(Course.SERVLETS);
System.out.println(Course.SPRING);
Course c2=Course.SPRING;
c2.setFacultyName("Dande");
c2.setDuration(3);
c2.setFee(5000);
System.out.println(c2);
} }
enum Course {
JAVA("DK",2,3000.0),JDBC("Sri",1,4000.0),SERVLETS("
Nivas",1,4000),SPRING;
private String facultyName;
private int duration;
private double fee;
Course(){}
private Course(String facultyName, int duration, double
fee) {
this.facultyName = facultyName;
this.duration = duration;        this.fee = fee;
}
public void setFacultyName(String facultyName) {
 this.facultyName = facultyName;
}
public void setDuration(int duration) {
 this.duration = duration;
}
public void setFee(double fee) {
 this.fee = fee;
}
public String toString() {
return
ordinal()+"\t"+name()+"\t"+facultyName+"\t"+duration+"
\t"+fee;
}
}
```

Lab1359.java

```
public class Lab1359 {
public static void main(String[] args){
Direction d1=null;
//d1=new Direction(0);
d1=Direction.EAST;
System.out.println(d1);
System.out.println(Direction.NORTH);
System.out.println(Direction.WEST);
System.out.println(Direction.SOUTH);
} }
enum Direction{
EAST,NORTH,WEST,SOUTH;
Direction(){
System.out.println(" Direction() Def Cons** ");
}
static{
System.out.println("** Static Block in Direction class");
}
}
```

- We can declare enum inside a class.
- Enums declared inside the class are always static and can be accessed as <OutClassName>.<EnumType>.<enumVar>.

```
public class TestOuter{
  enum Direction {
     EAST, WEST, NORTH,SOUTH
  }
}
```

- We can access a direction using TestOuter.Direction.NORTH.
- public final int ordinal(): To acces the integer value assigned for enum.
- public static >enumType>[] values() : To access all the enum constants defined in the enum.
- public static <enumType> valueOf(String name) : To verify that the enum has the constant with specified name or not.
- Note: If Constant not found with specified name then IllegalArgumentException will be thrown.

Lab1360.java

```
public class Lab1360 {
public static void main(String[] args){
Direction arr[]=Direction.values();
for(Direction d:arr){
System.out.println(d.ordinal()+"\t"+d.name());
}
try{
Direction d=Direction.valueOf("east");
System.out.println("** Direction found with the
name ");
}catch(IllegalArgumentException ex){
System.out.println("** No direction found with the
name ");
}
Direction d=Direction.valueOf("EAST");
System.out.println(d);
}
}
```

```
enum Direction{
EAST(0),NORTH(90),WEST(180),SOUTH(270);
int angle;
Direction(int angle){
this.angle = angle;
System.out.println(" Direction() Def Cons** ");
}
static{
        System.out.println("** Static Block in Direction
class\n");
}
}
```

- Enum constant can be used in switch statement.
- Enum constructors are by default private. You can use only private modifier with enum constructor.
- You can not invoke explicit super() constructor from Enum constructor.

Lab1361.java

```
public class Lab1361 {
public static void main(String[] args){
Direction d=Direction.EAST;
switch(d){
case EAST:
System.out.println("East Direction Selected"); break;
case NORTH:
System.out.println("North Direction Selected");
break;
case WEST:
System.out.println("West Direction Selected"); break;
case SOUTH:
System.out.println("South Direction Selected"); break;
}
}
}
enum Direction{
EAST,NORTH,WEST,SOUTH
}
```

```
enum Direction
EAST,NORTH,WEST,SOUTH;
public Direction(){}
}
```

Invalid

```
enum Direction{
EAST,NORTH,WEST,SOUTH;
Direction(){
 super();
}
}
```

Invalid

```
enum Direction{
EAST,NORTH,WEST,SOUTH;
abstract void show();
}
```

Invalid - method not overridden

- You can define an abstract method in enum. When you define an abstract method in enum then you need to override this method in all the constants.

Lab1362.java

```
public class Lab1362 {
public static void main(String[] args){
Direction.EAST.show();
Direction.WEST.show();
Direction.NORTH.show();
Direction.SOUTH.show();
}
}
```

```
enum Direction{
EAST(){
void show(){
System.out.println("showing East Direction");
}
},NORTH{
void show(){
System.out.println("showing North Direction");
}
},WEST{
void show(){
System.out.println("showing West Direction");
}
},SOUTH{
void show(){
System.out.println("showing South Direction");
}
};
abstract void show();
}
```

Lab1363.java

```
import java.util.*;
public class Lab1363 {
public static void main(String[] args) {
Set<Direction> set = EnumSet.allOf(Direction.class);
System.out.println(set);
Map<Direction, Integer> map = new
EnumMap<Direction, Integer>(Direction.class);
map.put(Direction.EAST,0);
map.put(Direction.NORTH,90);
map.put(Direction.WEST,180);
map.put(Direction.SOUTH,270);
System.out.println(map);
}
}
enum Direction {
EAST,NORTH,WEST,SOUTH
}
```

Points to Remember:

- enums are implicitly final subclasses of java.lang.Enum.
- If an enum is a member of a class, it's implicitly static.
- new can never be used with an enum, even within the enum type itself
- name() and valueOf() simply use the text of the enum constants.
- For enum constants, equals() and == operator does the same.
- enum constants are implicitly public static final.
- Constructors for an enum type should be declared as private.
- Since these enum instances are all effectively singletons, they can be compared for equality using identity (==).
- You can use Enum in Java inside Switch statement like int or char primitive data type.

Difference b/w Enum & Class

Class	Enum
By default for all the classes java.lang.Object is the super type.	By default for every enum java.lang.Enum is the super type.
Class can extends another class.	enum cannot extends another enum.
For class you can create object.	For enum instantiation is not allowed but you can declare reference variable.

Similarity:-

- ✓ Both can have the following members:
  - ❖ instance variable
  - ❖ static variable
  - ❖ instance block
  - ❖ static block
  - ❖ instance method
  - ❖ static method
  - ❖ constructors
  - ❖ abstract methods
- ✓ Both can implements interfaces.
- ✓ Class can have inner classes and Enum can also have inner enum.